

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2-DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	SHRUTHI K.	Reg.No.:	192571019
Department:	BTECH CSE & BIOSCIENCE	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____ (Name / Reg No)
 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					

Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Q1.

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

```
Q1 - Signed Integer Overflow (2's Complement) - code editor

1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main() {
5     // --- BUGGY: no overflow check ---
6     int8_t a = 120, b = 50;
7     int8_t buggy_sum = a + b;
8     printf("BUGGY: sum(120,50) as int8 = %d <-- wrong, wrapped around\n", buggy_sum);
9
10    // --- FIXED: overflow detection ---
11    int sign_a = (a >> 7) & 1, sign_b = (b >> 7) & 1, sign_r = (buggy_sum >> 7) & 1;
12    int overflow = (sign_a == sign_b) && (sign_r != sign_a);
13    printf("FIXED: overflow_flag = %d -> %s\n", overflow, overflow ? "OVERFLOW DETECTED" : "OK");
14    if (overflow) {
15        int16_t correct = (int16_t)a + (int16_t)b;
16        printf("FIXED: correct widened result = %d\n", correct);
17    }
18    return 0;
19 }
```

```
Terminal - GCC compile & run

$ gcc q1.c -o q1 && ./q1
BUGGY: sum(120,50) as int8 = -86 <-- wrong, wrapped around
FIXED: overflow_flag = 1 -> OVERFLOW DETECTED
FIXED: correct widened result = 170
```

Q2.

A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Q2 - Ripple Carry vs Carry Look-Ahead Adder - code editor

```
1 #include <stdio.h>
2 #define N 8
3 void ripple_carry_adder(int a[], int b[], int sum[], int *delay) {
4     int carry = 0; *delay = 0;
5     for (int i = 0; i < N; i++) {
6         sum[i] = a[i]^b[i]^carry;
7         carry = (a[i]&b[i]) | (carry&(a[i]^b[i]));
8         *delay += 2;           // BUG: delay accumulates -> O(N) bottleneck
9     }
10 }
11 void carry_lookahead_adder(int a[], int b[], int sum[], int *delay) {
12     int G[N], P[N], C[N+1]; C[0]=0;
13     for (int i=0;i<N;i++){ G[i]=a[i]&b[i]; P[i]=a[i]^b[i]; }
14     for (int i=0;i<N;i++){
15         int term=0;
16         for (int j=0;j<=i;j++){ int p=G[j]; for(int k=j+1;k<=i;k++) p&=P[k]; term|=p; }
17         C[i+1]=term;
18     }
19     for (int i=0;i<N;i++) sum[i]=P[i]^C[i];
20     *delay = 3;           // FIX: constant-depth parallel carry logic
21 }
22 int main() {
23     int a[N]={1,1,1,1,1,1,1,0}, b[N]={1,0,0,0,0,0,0,0};
24     int s1[N], s2[N], d1, d2;
25     ripple_carry_adder(a,b,s1,&d1);
26     carry_lookahead_adder(a,b,s2,&d2);
27     printf("RCA delay = %d units (scales with N)\n", d1);
28     printf("CLA delay = %d units (~O(1))\n", d2);
29     printf("Speed-up = %.2fx\n", (double)d1/d2);
30     return 0;
31 }
```

Terminal - GCC compile & run

```
$ gcc q2.c -o q2 && ./q2
RCA delay = 16 units (scales with N)
CLA delay = 3 units (~O(1))
Speed-up = 5.33x
```

Q3.

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

```
Q3 - Booth's Algorithm (Signed Multiplication) - code editor

1 #include <stdio.h>
2 #include <stdint.h>
3 #define N 8
4
5 // BUGGY: logical (unsigned) shift loses sign bit
6 int32_t booth_buggy(int32_t M, int32_t Q, int bits){
7     int32_t A=0, Qm1=0, Qreg=Q&((1<<bits)-1); int mask=(1<<bits)-1;
8     for(int i=0; i<bits; i++){
9         int q0=Qreg&1;
10        if(q0==1&&Qm1==0) A-=M; else if(q0==0&&Qm1==1) A+=M;
11        int64_t comb=((int64_t)(uint32_t)A<<bits)|Qreg;
12        Qm1=Qreg&1;
13        comb=(uint64_t)comb>>1; // BUG: logical shift
14        Qreg=comb&mask; A=(comb>>bits)&mask;
15    }
16    return (A<<bits)|Qreg;
17 }
18 // FIXED: arithmetic (sign-preserving) shift
19 int32_t booth_fixed(int32_t M, int32_t Q, int bits){
20     int32_t A=0, Qm1=0, Qreg=Q&((1<<bits)-1); int mask=(1<<bits)-1;
21     for(int i=0; i<bits; i++){
22         int q0=Qreg&1;
23         if(q0==1&&Qm1==0) A=A-M; else if(q0==0&&Qm1==1) A=A+M;
24         int32_t Ax=A; if(Ax&(1<<(bits-1))) Ax|=-mask; // sign-extend
25         int64_t comb=((int64_t)Ax<<bits)|Qreg;
26         Qm1=Qreg&1;
27         comb=comb>>1; // FIX: arithmetic shift
28         Qreg=comb&mask; A=(comb>>bits)&mask;
29         if(A&(1<<(bits-1))) A|=-mask;
30     }
31     int32_t p=(A<<bits)|(Qreg&mask);
32     int32_t m2=(1<<(2*bits))-1; p&=m2; if(p&(1<<(2*bits-1))) p|=-m2;
33     return p;
34 }
35 int main(){
36     int32_t M=-6, Q=5;
37     int32_t buggy=booth_buggy(M, Q, N);
38     int32_t fixed=booth_fixed(M, Q, N);
39     printf("Expected M*Q      = %d\n", M*Q);
40     printf("Booth BUGGY result = %d <-- wrong (logical shift corrupts sign)\n", (int16_t)(buggy&0xFFFF));
41     printf("Booth FIXED result  = %d <-- correct (arithmetic shift)\n", fixed);
42     return 0;
43 }
```

```
Terminal - GCC compile & run

$ gcc q3.c -o q3 && ./q3
Expected M*Q      = -30
Booth BUGGY result = 994 <-- wrong (logical shift corrupts sign)
Booth FIXED result = -30 <-- correct (arithmetic shift)
```

Q4.

An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Q4 - Restoring vs Non-Restoring Division - code editor

```
1 #include <stdio.h>
2 #define N 8
3 void restoring(int dividend,int divisor,int *Qo,int *Ro,int *ops){
4     int Q=dividend&((1<<N)-1), M=divisor, A=0; *ops=0;
5     for(int i=0;i<N;i++){
6         A=(A<<1)|((Q>>(N-1))&1); Q=(Q<<1)&((1<<N)-1);
7         A-=M; (*ops)++;
8         if(A<0){ A+=M; (*ops)++; Q&=~1; } // restore step = extra op (inefficient)
9         else Q|=1;
10    }
11    *Qo=Q; *Ro=A;
12 }
13 void nonrestoring(int dividend,int divisor,int *Qo,int *Ro,int *ops){
14     int Q=dividend&((1<<N)-1), M=divisor, A=0; *ops=0;
15     for(int i=0;i<N;i++){
16         A=(A<<1)|((Q>>(N-1))&1); Q=(Q<<1)&((1<<N)-1);
17         if(A>=0) A-=M; else A+=M; // FIX: 1 op/bit, no restore
18         (*ops)++;
19         if(A>=0) Q|=1; else Q&=~1;
20    }
21    if(A<0) A+=M;
22    *Qo=Q; *Ro=A;
23 }
24 int main(){
25     int q1,r1,o1,q2,r2,o2;
26     restoring(45,6,&q1,&r1,&o1);
27     nonrestoring(45,6,&q2,&r2,&o2);
28     printf("Restoring:      Q=%d R=%d ops=%d\n",q1,r1,o1);
29     printf("Non-Restoring: Q=%d R=%d ops=%d <-- fewer ops, same correct result\n",q2,r2,o2);
30     return 0;
31 }
```

Terminal - GCC compile & run

```
$ gcc q4.c -o q4 && ./q4
Restoring:      Q=7 R=3 ops=13
Non-Restoring: Q=7 R=3 ops=8 <-- fewer ops, same correct result
```

Q5.

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

```
Q5 - IEEE 754 Floating Point Precision - code editor

1 #include <stdio.h>
2 int main(){
3     // BUGGY: single precision loses the small term entirely
4     float big=1.0e8f, small=0.0001f;
5     float sum_f = big+small;
6     printf("float big+small = %.10f <-- 'small' vanished (23-bit mantissa)\n", sum_f);
7
8     // FIXED: double precision keeps far more of the small value
9     double big_d=1.0e8, small_d=0.0001;
10    double sum_d = big_d+small_d;
11    printf("double big+small = %.10f <-- 52-bit mantissa preserves it\n", sum_d);
12
13    // OPTIMIZATION: sum small terms together before adding to the large term
14    float vals[5]={1.0e8f,0.0001f,0.0001f,0.0001f,0.0001f};
15    float naive=0; for(int i=0;i<5;i++) naive+=vals[i];
16    float small_sum=vals[1]+vals[2]+vals[3]+vals[4];
17    float reordered=vals[0]+small_sum;
18    printf("naive left-to-right sum   = %.10f\n", naive);
19    printf("small-terms-first sum    = %.10f\n", reordered);
20    return 0;
21 }
```

```
Terminal - GCC compile & run

$ gcc q5.c -o q5 && ./q5
float big+small = 100000000.0000000000 <-- 'small' vanished (23-bit mantissa)
double big+small = 100000000.0001000017 <-- 52-bit mantissa preserves it
naive left-to-right sum   = 100000000.0000000000
small-terms-first sum    = 100000000.0000000000
```